DEVFLOW CODEBASE

Design Patterns Architecture Report

A comprehensive documentation of structural, creational, and behavioral design patterns implemented within the DevFlow backend application.

Generated On: June 13, 2026

Target System: DevFlow Backend (Python/Django & MongoEngine)

Format: Developer Technical Reference Document (PDF)

Executive Summary & Patterns Index

This document details the architectural design patterns implemented across the DevFlow backend codebase. Design patterns improve system maintainability, readability, and scalability. The DevFlow backend adheres to established enterprise software engineering patterns, facilitating structured state flows, dynamic prioritize handling, and clean notification propagation. Below is the list of documented patterns:

Pattern Name	Implementation File	Type
Singleton Pattern	core/singleton.py	Creational
Strategy Pattern	tasks/strategy.py	Behavioral
State Pattern	tasks/state.py	Behavioral
Builder Pattern	reports/builder.py	Creational
Observer Pattern (Core)	core/observers.py	Behavioral
Observer Pattern (Notifications)	notifications/subject.py & observer.py	Behavioral
Chain of Responsibility Pattern	core/approval_chain.py	Behavioral
Command Pattern	core/commands.py	Behavioral
Factory Pattern	core/factories.py	Creational

Singleton Pattern

File Reference: core/singleton.py

The Singleton pattern ensures a class has only one instance and provides a global point of access to it. In DevFlow, this is used for the database connection (MongoConnection) to ensure we don't open duplicate connections to MongoDB.

Implementation Code:

```
class MongoConnection:

    _instance = None

    @staticmethod
    def get_instance():

        if MongoConnection._instance is None:
            MongoConnection._instance = MongoConnection()

        return MongoConnection._instance
```

Strategy Pattern

File Reference: tasks/strategy.py

The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. Here, it encapsulates different task priority execution strategies (Low, Medium, High Priority) and runs them dynamically based on context.

Implementation Code:

```
class LowPriorityStrategy:
    def execute(self):
        return "Low Priority Task"

class MediumPriorityStrategy:
    def execute(self):
        return "Medium Priority Task"

class HighPriorityStrategy:
    def execute(self):
        return "High Priority Task"

class PriorityContext:
    def __init__(self, strategy):
        self.strategy = strategy
    def run(self):
        return self.strategy.execute()
```

State Pattern

File Reference: tasks/state.py

The State pattern allows an object to alter its behavior when its internal state changes. In DevFlow, task status transitions (Todo -> InProgress -> Testing -> Done) are governed by state objects, making state transitions clean and preventing invalid transitions.

Implementation Code:

```
class TodoState:
    def next(self):
        return "InProgress"

class InProgressState:
    def next(self):
        return "Testing"

class TestingState:
    def next(self):
        return "Done"

class DoneState:
    def next(self):
        return "Done"
```

Usage Context: State Pattern Usage in tasks/views.py (ChangeTaskStatusView)

```
        from .state import TodoState, InProgressState, TestingState, DoneState
        state_map = {
            "Todo": TodoState(),
            "InProgress": InProgressState(),
            "Testing": TestingState(),
            "Done": DoneState()
        }
        current_state_str = task.status or "Todo"
        state_inst = state_map.get(current_state_str, TodoState())
        next_state = state_inst.next()
        task.status = next_state
        task.save()
```

Builder Pattern

File Reference: reports/builder.py

The Builder pattern separates the construction of a complex object from its representation. In DevFlow, ReportBuilder builds a report document step-by-step by accumulating counts for projects, sprints, tasks, and bugs, and then returning the final report.

Implementation Code:

```
class Report:

    def __init__(self):

        self.data = {}

    def add(self, key, value):

        self.data[key] = value

    def build(self):

        return self.data


class ReportBuilder:

    def __init__(self):

        self.report = Report()

    def project_count(self, count):

        self.report.add(
            "total_projects",
            count
        )

    def sprint_count(self, count):

        self.report.add(
            "total_sprints",
            count
        )

    def task_count(self, count):

        self.report.add(
            "total_tasks",
            count
        )

    def bug_count(self, count):

        self.report.add(
            "total_bugs",
            count
        )

    def get_report(self):

        return self.report.build()
```

Usage Context: Builder Pattern Usage in reports/services.py (ReportService)

```
class ReportService:

    @staticmethod
    def generate_report():

        builder = ReportBuilder()

        builder.project_count(
            Project.objects.count()
        )

        builder.sprint_count(
            Sprint.objects.count()
        )

        builder.task_count(
            Task.objects.count()
        )

        builder.bug_count(
            Bug.objects.count()
        )

        return builder.get_report()
```

Observer Pattern (Core)

File Reference: core/observers.py

The Observer pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically. In DevFlow, the Subject class manages observers like EmailObserver and NotificationObserver and broadcasts messages to them.

Implementation Code:

```
class Observer:

    def update(
        self,
        message
    ):
        pass


class EmailObserver(
    Observer
):

    def update(
        self,
        message
    ):

        print(
            f"Email Sent: {message}"
        )


class NotificationObserver(
    Observer
):

    def update(
        self,
        message
    ):

        print(
            f"Notification Sent: {message}"
        )


class Subject:

    def __init__(self):

        self.observers = []

    def attach(
        self,
        observer
    ):

        self.observers.append(
            observer
        )

    def notify(
        self,
        message
    ):

        for observer in self.observers:

            observer.update(
                message
            )
```

Observer Pattern (Notifications)

File Reference: notifications/subject.py & observer.py

A domain-specific implementation of the Observer pattern for the notification system. The NotificationSubject tracks observers and notifies them, print-formatting notification events when triggered.

Implementation Code:

```
class NotificationSubject:

    def __init__(self):

        self.observers = []

    def attach(self, observer):

        self.observers.append(
            observer
        )

    def notify(self, message):

        for observer in self.observers:

            observer.update(
                message
            )
```

Observer Implementation Code: notifications/observer.py

```
class Observer:

    def update(self, message):
        pass


class NotificationObserver(
    Observer
):

    def update(self, message):

        print(
            f"Notification: {message}"
        )
```

Chain of Responsibility Pattern

File Reference: core/approval_chain.py

The Chain of Responsibility pattern passes a request along a chain of handlers. Each handler decides either to process the request or to pass it to the next handler in the chain. In DevFlow, project approval requests pass from PMApproval to AdminApproval.

Implementation Code:

```
class Handler:

    def __init__(self):

        self.next_handler = None

    def set_next(
        self,
        handler
    ):

        self.next_handler = handler

        return handler


class PMApproval(
    Handler
):

    def handle(
        self,
        request
    ):

        if request == "Project":

            return (
                "Approved By PM"
            )

        if self.next_handler:

            return self.next_handler.handle(
                request
            )


class AdminApproval(
    Handler
):

    def handle(
        self,
        request
    ):

        return (
            "Approved By Admin"
        )
```

Command Pattern

File Reference: core/commands.py

The Command pattern encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations. DevFlow uses it to encapsulate task actions (CreateTaskCommand, DeleteTaskCommand).

Implementation Code:

```
class Command:

    def execute(self):
        pass


class CreateTaskCommand(
    Command
):

    def execute(self):

        return (
            "Task Created"
        )


class DeleteTaskCommand(
    Command
):

    def execute(self):

        return (
            "Task Deleted"
        )
```

Factory Pattern

File Reference: core/factories.py

The Factory pattern provides an interface for creating objects in a superclass, but allows subclasses or specific factory logic to alter the type of objects that will be created. In DevFlow, UserFactory creates and saves User instances with specific roles.

Implementation Code:

```
from accounts.models import User


class UserFactory:

    @staticmethod
    def create_user(data):

        role = data["role"]

        user = User(

            name=data["name"],

            email=data["email"],

            password=data["password"],

            role=role
        )

        user.save()

        return user
```